

CMPE260 Basic Interpreter Design Project

Efe Berke Vatansever - 2025402006

1 Scoping Decision

I implemented static scoping so that whenever a variable is searched (lookup() in environment.py), the interpreter first searches the current local environment and then recursively searches parent environments. This environment-chain structure produces static (lexical) scoping behavior.

In dynamic scoping, the interpreter would instead follow the runtime call stack. For example, if a function x is called from another function y and the interpreter searches for a variable a, it would first search in the local scope of x, then continue searching in the caller function y, and so on through the call stack.

I chose static scoping because it is more readable, and its major advantage is that programs can be analyzed without running them, simply by looking at the code itself. It increases interpreter complexity slightly because of environment capturing and parent environment management. In simple and small programs, static scoping feels like a more straightforward choice compared to the runtime analysis required in dynamic scoping.

Demonstration of Static and Dynamic Scoping

```
let x = 100;

let inner = fun() ->
  print(x);
end;

let outer = fun() ->
  let x = 10;
  inner();
end;

outer();
```

Under static scoping (my choice), the program eventually reaches the inner function, and when it comes to print(x), it reads the value from its parent environment because it does not have a local x. Therefore, the output will be 100.

If we consider dynamic scoping, when execution reaches inner, it was previously called from outer,

and outer has a variable `x` assigned. Therefore, inner reads `x` from the outer function, so the output will be 10.

2 Evaluation Strategy

Demonstration of Call-by-Value

```
let add = fun(a, b) ->
  a + b
end;

add(3 + 1, 2 * 5);
```

My interpreter uses call-by-value, as required in the project description. It evaluates arguments before the function receives them, so in the example, the function directly receives 4 and 10 as `a` and `b`. Call-by-value is much simpler compared to call-by-name. In the above example, call-by-name produces the same result, but it computes the values of `a` and `b` much later than call-by-value languages.

Clear Comparison of Call-by-Value and Call-by-Name

```
let f = fun(x) ->
  1
end;

f(5/0)
```

In this situation, call-by-value languages directly evaluate `5 / 0`, and a runtime error occurs immediately. However, in call-by-name languages, `x` is never used inside the function, so `5 / 0` is never evaluated, and the function successfully returns 1.

3 Closure Representation

Closures in my interpreter are implemented using the `FunctionValue` object. "params" is for parameter names of the function, "body" is for AST block representing the function body. Finally, "env" is for environment where the function was defined. The captured "env" is the key part for the closure. It allows the function to remember variables from its defining scope even if the function is called later from another place.

```
class FunctionValue:
  def __init__(self, params, body, env):
    self.params = params
    self.body = body
    self.env = env
```

4 Environment Management

```
class Environment:

    def __init__(self, parent = None): ...

    # defining new variable
    def define(self, name, value): ...

    # finding the variable value
    def lookup(self, name): ...

    # updating the existing variable
    def assign(self, name, value): ...
```

Each environment frame stores its own local variables inside a dictionary called `values` and also keeps a reference to its parent environment. The `values` dictionary stores the bindings of the current scope, while the parent reference connects the current environment to the outer scope. This creates the actual environment chain.

We have `define`, `lookup`, and `assign` functions in the `Environment` class.

define is used for creating the first binding of a variable. It binds a variable name to a value in the current environment frame.

lookup is used for finding variables. It first searches the current local environment, and if the variable is not found, it recursively searches parent environments.

assign is used for updating an existing binding between a variable and its value. It first checks the current environment, and if the variable does not exist there, it continues searching through parent environments until the variable is found and updated.

Tracing the following program under static scoping:

```
let x = 1;
let f = fun(a) ->
  let y = a + x;
  fun() ->
    x = x + 1;
    y + x
  end
end;
let g = f(10);
print(g());
print(g());
```

Initial Frames

```
Global Environment: G
-----
x -> 1
f -> FunctionValue
```

Before "let g = f(10);" only global frame exists and just x and function is defined in the frame.

Call to f(10)

```
Call frame for f(10): F1
Parent: G
-----
a -> 10
y -> 11

Global Environment: G
-----
x -> 1
f -> FunctionValue(env = G)
g -> FunctionValue(env = F1)
```

First Call to g()

```
Frame: F1
Parent: G
-----
a -> 10
y -> 11

Global Environment: G
-----
x -> 2
f -> FunctionValue(env = G)
g -> FunctionValue(env = F1)

Frame: G1
Parent: F1
-----
(no local bindings)
```

Second Call to g()

```
Frame: F1
Parent: G
-----
a -> 10
y -> 11

Global Environment: G
-----
x -> 3
```

```
f -> FunctionValue(env = G)
g -> FunctionValue(env = F1)
```

```
Frame: G2
Parent: F1
```

```
-----
(no local bindings)
```

G1 and G2 are destroyed after being called. F1 survives because g closure captures F1(g stores reference to F1 in global frame.).

In each g() call inner function activates and update the value of x. This is why in each new global frame, x is incremented by 1.

5 Grammar Ambiguity

I handle grammar ambiguity by separating expressions into precedence levels in parser.py. The precedence order is designed as follows:

```
parse_or()
parse_and()
parse_comparison()
parse_expression()    -> +, -
parse_term()          -> *, /
parse_call()
parse_factor()
```

This structure removes ambiguity. For example, if we write: "5 * 7 - 8" it is parsed as: "(5 * 7) - 8" because the parse term function handles * before parse expression function handles -.

I also use the end keyword for every if statement, which removes ambiguity in conditional expressions.

In addition, assignment statements are detected with the following condition in parser.py:

```
if token.type == "IDENTIFIER" and self.peek().type == "EQUAL":
    return self.parse_assignment()
```

This means that if an identifier is followed by =, it is parsed as an assignment statement. Otherwise, it is parsed as an expression.